

Lab 3: Architectural Justification

Student Club Event Ticketing & Budget Portal | PES1UG24CS498

Architecture Selection

We chose Layered Architecture for the Student Club Event Ticketing & Budget Portal.

Architectural Choice

We went with a three-layer style: a Presentation layer (User Interface Component), a Business layer (Proposal & Approval Manager, Ticketing Service and Check-in Service components) and a Data layer (Database Component). Each layer only talks to the one directly below it through a defined interface, which is what the component diagram shows. We compared it with Microservices and plain Client-Server before deciding, and Layered fit this scenario best.

Two Reasons

1. The workflow is sequential and rule-driven. A budget proposal has to pass the Faculty Coordinator, then the Finance Office, then the Dean, and a ticket goes from registration to QR generation to scanning at the venue. This maps directly onto UI → business logic → database. Keeping the approval rules (FR-001) in one Business-layer component means the order of sign-offs is enforced in one place and the interface screens cannot skip a stage.

2. It suits the size of the system and the team. This is a campus tool with moderate traffic that spikes around event registration and entry. Microservices would mean separate deployments, service discovery and data split across services, which is too much overhead for a student team and makes things like duplicate-entry detection (FR-004) harder to keep consistent. With layers there is one deployable system and one database, so budgets, tickets and check-ins always agree with each other.

Security Advantage

Only the Business layer can reach the Database Component, and the UI never queries data directly. Role checks (Club Lead, Approver, Campus Admin) and QR validation therefore happen on the server side, behind the interfaces, where they cannot be bypassed from the screen. Encryption of attendee data (HTTPS in transit and AES-256 at rest, NFR-002) is also applied once at the Data layer boundary instead of being repeated in every component.

Performance Benefit

The Check-in Service validates a scanned QR code with a single call to the central database inside the same deployment, so there are no extra network hops between separate services. That helps us stay under the 100 ms validation target in NFR-001 at the venue entrance. The trade-off is that every layer adds a little overhead and the database can become a bottleneck at peak entry time, which we would handle by indexing the ticket ID column and caching recent lookups in the Business layer.